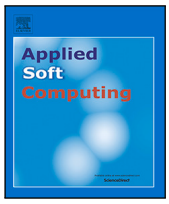
Applied Soft Computing Journal 171 (2025) 112787

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc



Dynamic scheduling for multi-objective flexible job shop via deep reinforcement learning

Erdong Yuan [a], Liejun Wang [a,*], Shiji Song [b], Shuli Cheng [a], Wei Fan [c]

[a] School of Computer Science and Technology, Xinjiang University, Urumqi, 830017, Xinjiang, China
[b] Department of Automation, Tsinghua University, 100084, Beijing, China
[c] AI Laboratory, Lenovo Research, 100085, Beijing, China

ARTICLE INFO

Keywords:
Flexible job shop scheduling problem
Deep reinforcement learning
Dynamic multi-objective
Real-time scheduling
Generalization

ABSTRACT

The dynamic scheduling methods for multi-objective flexible job shop are in high demand for actual production. Deep reinforcement learning (DRL) can realize offline training and online solving, which has become a promising method for dynamic multi-objective flexible job shop scheduling problem (DMOFJSSP). The dynamic event we focus on is new job insertions. In this paper, we first propose a new Markov decision process model to enhance the learning effect of the algorithm, where the new state representation is designed according to the dynamic and multi-objective characteristics of the problem, which helps the agent to capture more comprehensive state information and improve its decision-making ability; the new action space is composed of operation-machine pairs combined with invalid action masking technology to effectively explore the solution space; the two new reward functions are designed, which are closely related to multi-objective optimization, and both can effectively guide the agent to recognize key actions. Next, we propose a lightweight network architecture to realize representation learning and policy learning, which reduces the computational complexity of the algorithm to some extent. Finally, we evaluate the generalization of the policy model on two test datasets from different literatures. Experimental results show that the proposed method can achieve better optimization results compared with the well-known composite priority dispatching rules and the state-of-the-art models. The implementation code for the proposed method is available at https://github.com/cslxju/DMOFJSSP_DRL.

1. Introduction

With the progress of society, people's demand for products is becoming more and more abundant. In workshop production, the scheduling process is often faced with various situations [1], such as multi-variety, small batch, order increase or decrease, and machine breakdown. Dynamic multi-objective flexible job shop scheduling problem (DMOFJSSP) is extended on basis of the static single-objective flexible job shop scheduling problem (FJSSP), which has been proved to be NP-Hard [2,3]. For FJSSP, it is necessary to solve not only the operation sequencing sub-problem, but also the machine allocation sub-problem [4]. In the industry, the single-objective FJSSP is often difficult to meet the actual needs of production, so dispatchers pay more attention to multi-objective optimization [5]. Due to the increasing complexity and uncertainty of scheduling, there is an urgent need to study some effective methods to solve DMOFJSSP.

The traditional research methods for solving the dynamic shop scheduling problem include three types [6,7]: completely reactive scheduling, predictive-reactive scheduling, and robust pro-active scheduling. During scheduling, completely reactive scheduling has strong real-time performance and can realize fast decision making without generating a scheduling scheme in advance. This type of method mainly includes some priority dispatching rules (PDRs), which are easy to make decisions through some local information. However, they have the defect of short-sightedness and are difficult to predict system performance. Predictive-reactive scheduling is often used in dynamic scheduling, which presents a scheduling/rescheduling process. When a new dynamic event occurs, they will update the original scheduling scheme to cope with the new production scenario. This type of method first converts complex dynamic scheduling problems into a series of static scheduling problems, and then uses some methods for solving these static scheduling problems, such as meta-heuristic methods. Meta-heuristic methods are mainly some iterative search algorithms, which are often inspired by natural phenomena. The design of this type of method is complicated, and the quality of the solution is unstable due to the presence of random factors in the algorithm design. Besides, it is difficult to cope with environments where scheduling plans are frequently updated. Currently, there is relatively little research on robust

* Corresponding author.
E-mail address: wljxju@xju.edu.cn (L. Wang).

https://doi.org/10.1016/j.asoc.2025.112787
Received 2 December 2023; Received in revised form 13 September 2024; Accepted 17 January 2025
Available online 25 January 2025

pro-active scheduling that predicts the occurrence of some uncertain events in advance, and it takes certain measures to deal with various uncertainties before a dynamic event occurs. For example, additional time can be inserted into the schedule in advance to reduce deviations from the original schedule and thereby enhance its robustness. While these measures can accommodate some uncertainty and make the scheduling process more robust, they do so at the expense of response time.

In recent years, deep reinforcement learning (DRL) has become a promising new method to solve various shop scheduling problems as a solver [8]. This method can achieve offline training and online solving, i.e. training a policy model offline and using its generalization to solve other scale problems online. However, it still has shortcomings in state representation, action space definition and reward function design for Markov Decision Process (MDP) modeling, which are not conducive to the agent learning an effective policy.

The research motivation of this work is to learn an effective policy to solve DMOFJSSP via DRL, whose minimization optimization objectives include makespan, mean lateness and mean flow time. Previous DRL methods solve this problem by selecting appropriate PDRs. However, the short-sightedness of the PDRs makes it difficult for the agent to explore the solution space effectively. This paper aims to improve state representation, action space definition and reward design when modeling MDP. At the same time, a lightweight network architecture is used to implement representation learning and policy learning.

The primary contributions of this study are summarized as follows:

- We propose a lightweight network architecture to implement representation learning and policy learning, which reduces the computational complexity of the algorithm to some extent.
- We design a new MDP model for DMOFJSSP to enhance the learning effect of the algorithm, in which the new state representation is designed according to the dynamic and multi-objective characteristics of the problem, which is beneficial for the agent to capture more comprehensive state information and improve its decision-making ability; the new action space is composed of operation-machine pairs, which helps the agent to better explore the solution space. In addition, two new reward functions are designed, which are closely related to multi-objective optimization, and both can effectively guide the agent to recognize key actions.
- In order to verify the effectiveness of the policy model, the proposed method evaluates the generalization of the model on two test datasets from different literatures. Experimental results show that the policy model trained by the proposed method has better optimization effects than the well-known composite PDRs and the current state-of-the-art models.

The rest of content is organized as follows: Section 2 provides the literature review; Section 3 describes DMOFJSSP; Section 4 introduces the implementation details of the proposed method; Section 5 presents numerical experiments; Conclusions are given in Section 6.

2. Literature review

In this section, we review three traditional methods and the learning-based method for solving dynamic FJSSP.

2.1. Completely reactive scheduling

Completely reactive scheduling is also a constructive method that generates a solution starting from no scheduling [9]. It is easy to implement and has the best real-time performance. However, it has the defect of short-sightedness, which often leads to the poor quality of the solution. Some composite PDRs [10] are often used to solve DMOFJSSP, which consists of operation sequencing rules and machine allocation rules. Some common sequencing rules for operations include *First In First Out* (FIFO), *Most Work Remaining* (MWKR) and *Most Operation Number Remaining* (MOPNR), etc. Some common machine allocation rules, such as *Shortest Processing Time* (SPT) and *Earliest End Time* (EET).

2.2. Predictive-reactive scheduling

Predictive-reactive scheduling is widely used to solve dynamic FJSSP, which presents a scheduling/rescheduling process. Shen et al. [11] proposed a predictive-reactive scheduling method based on ε-MOEA to solve DMOFJSSP, which belongs to an evolutionary algorithm. It can simultaneously optimize the related objectives of shop efficiency and stability. Gao et al. [12] proposed a two-stage artificial bee colony algorithm to solve dynamic FJSSP with new job insertions. The two stages refer to scheduling and rescheduling, respectively. Nouiri et al. [13] proposed a two-stage particle swarm optimization (2S-PSO) method for the flexible job shop predictive scheduling problem with only one machine breakdown. Li et al. [14] proposed a method for solving dynamic FJSSP based on Monte Carlo Tree Search (MCTS) [15] algorithm. This method uses time windows [16] to periodically update the original schedule and generates the corresponding subsequent partial schedule for the remaining unscheduled operations according to the schedule repair strategy [17]. An et al. [18] proposed an improved Non-Dominated Sorting Genetic Algorithm III (NSGA3) [19] with adaptive reference vector, which was used to solve the multi-objective FJSSP with new job insertions and machine preventive maintenance. Wei et al. [20] proposed a migratory bird optimization algorithm to solve DMOFJSSP with machine breakdown. The algorithm introduces game theory to balance the relationship between production efficiency and stability.

2.3. Robust pro-active scheduling

Robust pro-active scheduling aims to predict possible dynamic events in advance and take certain measures to deal with the interference of uncertain events during scheduling. Xiong et al. [21] proposed a robust evolutionary algorithm to solve DMOFJSSP, which considers the probability of machine breakdowns and the location of float times and machine breakdowns. Zhang et al. [22] proposed two robust goal-guided multi-objective search methods with flexible workdays to solve DMOFJSSP, whose optimization objectives include tardiness, overtime and robustness. Shen et al. [23] proposed a improved robust multi-objective evolutionary algorithm to solve the stochastic FJSSP, whose optimization objectives include makespan, maximal machine workload and robustness.

2.4. Dynamic scheduling via the learning-based method

The learning-based methods for solving dynamic FJSSP mainly include two research branches: Reinforcement learning (RL) method and Genetic programming (GP) method. The research branch of this paper mainly involves the DRL-based method. (1) RL method: The success of DRL in Atari [24], AlphaGo [25] and AlphaStar [26] has led researchers to hope that this technology can be used to solve some combinatorial optimization problems, such as FJSSP. DRL performs scheduling through offline training and online solving, so its rescheduling strategy is close to completely reactive scheduling. Luo et al. [27] developed a deep Q-network (DQN) to solve dynamic single-objective FJSSP with new job insertions. For MDP modeling, the author defines seven general state features and uses six composite PDRs to define the action space for the first time. Luo et al. [28] constructed a two-hierarchy deep Q network (THDQN) to solve DMOFJSSP with new job insertions, whose action space is composed of six composite PDRs. Luo et al. [29] also proposed a hierarchical multi-agent real-time scheduling method to solve the wait-free DMOFJSSP, whose action space consists of five operation sequencing rules and six machine allocation rules. Liu et al. [30]

proposed a hierarchical distributed architecture for solving dynamic FJSSP. The double DQN (DDQN) [31] training the policy model is used to capture the relationship between state information and optimization objectives. Wang et al. [32] trained the policy model using Dynamic Multi-Objective Double Deep Q-Learning Network (DMDDQN) to solve DMOFJSSP with multiple dynamic events in real time, whose optimization objectives include makespan, average machine utilization, and average job processing delay rate. The action space consists of nine combined dispatching rules. Chang et al. [33] adopted DDQN as the network architecture and designed a new MDP model to train the policy model for solving FJSSP with random arrival of jobs. Its action space consists of four composite PDRs. Zhang et al. [34] considered variable processing time as a dynamic event, and trained the policy model via DRL to solve dynamic FJSSP. The defined action space consists of eight hybrid scheduling rules (HSR). Gui et al. [35] used a deep deterministic policy gradient (DDPG) [36] algorithm to train the policy model for dynamic FJSSP. This method takes minimizing the average delay as the optimization objective, and its action space is a continuous rule space that combines dispatching rules and continuous weight variables. Liu et al. [37] proposed a method to train a deep Q network based on Genetic algorithm (GA), and adopted the model to solve dynamic single-objective FJSSP. Tang et al. [38] proposed a hybrid algorithm that combines Double Q-Learning with NSGA3 to solve the flexible job shop dynamic scheduling problem. This hybrid method uses Double Q-Learning to adjust the key parameters of NSGA3 to effectively improve its optimization capability. (2) GP method: GP can also achieve offline training and online solving, as RL, as well. Thus, some GP methods [39–43] are also used to solve dynamic FJSSP. Both GP and DRL methods have their own advantages and disadvantages in the examined scenarios [43], and GP is more advanced than a DRL method and has a better optimization capability with increasing training data, but the solution generated by GP is unstable.

Solving dynamic FJSSP via DRL not only focuses on the decision-making ability of the model, but also considers real-time performance. The action space defined by some current DRL methods usually consists of some PDRs. However, due to the short-sightedness of PDRs, the optimization capability of the policy model needs to be improved. At the same time, some works often train two sub-policy models to solve two sub-problems of dynamic FJSSP, which reduces real-time performance to some extent. This paper uses a lightweight network architecture to implement representation learning and policy learning. The proposed method designs a new MDP model, in which the new state representation is designed according to the dynamic and multi-objective characteristics of the problem, which is beneficial for the agent to capture more comprehensive state information and improve its decision-making capability; the new action space consists of operation-machine pairs, which helps the agent to better explore the solution space. In addition, the two newly designed reward functions are closely related to multi-objective optimization and help guide the agent to identify key actions.

3. Problem description for DMOFJSSP

DMOFJSSP is defined as follows: There are n jobs $\boldsymbol{J} = \{J_i\}_{1\le i\le n}$ that need to be processed on m machines $\boldsymbol{M} = \{M_k\}_{1\le k\le m}$. Each job contains at least one operation $O_{ij}\ (j = 1, \dots, n_i)$, and each operation can only select one machine from the optional machine set $\boldsymbol{\Omega_{ij}} \in \boldsymbol{M}$ for processing. During solving, the priority constraints between adjacent operations of the same job need to be met; the machine capacity constraints also need to be met, that is, each machine can only process one operation at a time. Some dynamic events often appear during production, such as the arrival of new order, order cancellation, and machine breakdown. The dynamic event involved in this paper is new job insertions. The three minimization optimization objectives include makespan, mean lateness and mean flow time. It is worth noting that minimizing makespan is equivalent to maximizing average machine utilization. Like FJSSP, it also needs to solve two sub-problems for solving DMOFJSSP, namely the operation sequencing sub-problem and the machine allocation sub-problem. Some assumptions for solving this problem are as follows:

(1) The transport time between operations can be ignored or included in the processing time;
(2) Each operation is not allowed to be interrupted during processing;

In order to represent DMOFJSSP more intuitively, the following symbols are defined:

(1) Parameters:
n: total number of jobs;
m: total number of machines;
i, h: index of jobs, $i, h = 1, \dots, n$;
n_i: the number of operations for J_i;
j, g: index of operations, $j, g = 1, \dots, n_i$;
k, e: index of machines, $k, e = 1, \dots, m$;
O_{ij}: the jth operation of J_i;
$\boldsymbol{\Omega_{ij}}$: the optional machine set for O_{ij};
O_{ijk}: the operation O_{ij} is processed on machine M_k;
p_{ijk}: the processing time of O_{ij} on machine M_k;
$\bar{p}_{ij}$: the average processing time of O_{ij} on $\boldsymbol{\Omega_{ij}}$;
s_{ij}: the start time of O_{ij};
c_{ij}: the completion time of O_{ij};
L: a sufficiently large positive number;
r_i: the arrival time of J_i;
d_i: the due date of J_i;
C_i: the completion time of J_i;
F_1: makespan;
F_2: mean lateness;
F_3: mean flow time;
(2) Decision variables:

$$x_{ijk} = \begin{cases} 1, & \text{if } O_{ij} \text{ selects processing machine } M_k; \\ 0, & \text{otherwise;} \end{cases}$$

$$y_{ijhgk} = \begin{cases} 1, & \text{if } O_{ijk} \text{ is a predecessor of } O_{hgk}; \\ 0, & \text{otherwise;} \end{cases}$$

3.1. Mathematical model

Combined with the model developed by [44], the mathematical model of DMOFJSSP is as follows:

Minimize:

$$\begin{cases} F_1 = \max\{C_i\} \\ F_2 = \dfrac{\sum_{i=1}^{n} \max\{C_i - d_i, 0\}}{n} \\ F_3 = \dfrac{\sum_{i=1}^{n} (C_i - r_i)}{n} \end{cases} \tag{1}$$

Subject to:

$$r_i + x_{i1k} \times p_{i1k} \le c_{i1} \tag{2}$$

$$s_{ij} + x_{ijk} \times p_{ijk} \le c_{ij} \tag{3}$$

$$c_{ij} \le s_{i(j+1)} \tag{4}$$

$$s_{ij} + p_{ijk} \le s_{hg} + L(1 - y_{ijhgk}) \tag{5}$$

$$c_{ij} \le s_{i(j+1)} + L(1 - y_{hgi(j+1)k}) \tag{6}$$

$$\sum_{k=1}^{|\boldsymbol{\Omega_{ij}}|} x_{ijk} = 1 \tag{7}$$

The three optimization objectives are shown in Eq. (1). Eqs. (2), (3) and (4) indicate the priority constraints to be satisfied between adjacent

Table 1
A dynamic FJSSP instance with 3 jobs and 4 machines.

Jobs	Arrival time	Due date	Operations	Optional machine and processing time			
				M_1	M_2	M_3	M_4
J_1	$r_1 = 17.69$	$d_1 = 42.79$	O_{11}	4.82	–	–	2.94
			O_{12}	–	–	3.47	4.13
			O_{13}	–	7.38	4.91	–
			O_{14}	2.70	–	3.13	–
J_2	$r_2 = 3.74$	$d_2 = 18.67$	O_{21}	–	–	2.88	3.63
			O_{22}	5.69	–	–	7.71
J_3	$r_3 = 11.12$	$d_3 = 26.04$	O_{31}	–	3.49	5.89	–
			O_{32}	–	5.70	–	4.82

operations of the same job. Eqs. (5) and (6) mean that the machine has limited processing capacity, and it can only process one operation at a time. Eq. (7) expresses that an operation can only be processed by one machine at a time.

3.2. Disjunctive graph representation

Disjunction graph (DG) is used to describe JSSP [45–47], and it can also be used to describe FJSSP. For FJSSP, DG $= (\mathcal{N}, \mathcal{A}, \mathcal{E})$ is represented as a DG model, where $\mathcal{N}$ represents a set of all operations, which contains two virtual nodes (a virtual start node 0 and a virtual end node $*$); $\mathcal{A}$ is a set of directed arcs (conjunctions) between adjacent operations on the same job, representing the priority constraints between operations; $\mathcal{E}$ is a set of undirected arcs (disjunctions), each of which connects a pair of operations that can be processed on the same machine. Different colors of the disjunctive arcs represent different machines.

Table 1 shows a dynamic FJSSP with 3 jobs and 4 machines. In this instance, each job has a random arrival time, which reflects the dynamic characteristic of the problem. It is worth noting that the instance needs to be preprocessed to ensure that each job has an equal number of operations, which facilitates the calculation of the estimated completion time of each operation by an array or tensor. The proposed method ensures that the number of operations of each job is equal to the maximum number of operations of all jobs by padding nodes, and sets the processing time of the padding nodes to 0. In addition, a masking restriction is applied to the padding nodes, i.e., the padding nodes do not participate in scheduling.

Fig. 1 shows the disjunction graph representation of the preprocessed DMOFJSSP instance, in which the masked padding nodes are marked with purple red slashes. These masked padding nodes do not participate in scheduling. In Fig. 1(a) represents the initial disjunction graph representation of a dynamic FJSSP instance. Each node has not yet been scheduled, and the processing time above it represents the average processing time of the corresponding operation on the optional machine set; Fig. 1(b) shows a complete solution, each node is scheduled, and the processing time above it represents the actual processing time of the corresponding operation on the selected machine. At the same time, the directions of all disjunctive arcs are also determined.

To better explain what the processing time above each node in Fig. 1 represents, we give an example. For example, $\bar{p}_{11} = 3.88$ means that the average processing time of operation O_{11} on the optional machine set $\boldsymbol{\Omega_{11}}$ is 3.88; $p_{111} = 4.82$ represents that the actual processing time of operation O_{11} on the selected machine M_1 is 4.82.

4. Methodology

Fig. 2 shows the DRL framework with reinforcement learning working mechanism. This framework mainly involves MDP modeling and model training, where model training includes representation learning and policy learning. It should be noted that the overall framework uses a lightweight network architecture. In Fig. 2, the initial state is represented as the state information of the initial jobs, and as new jobs are inserted, the update state is represented as the combined state information of the initial jobs and the new job insertions.

4.1. Markov decision process modeling

An MDP model mainly consists of a tuple $(\boldsymbol{S}, \boldsymbol{A}, \boldsymbol{P}, \boldsymbol{R}, \gamma)$. $\boldsymbol{S}$ is a set of all states; $\boldsymbol{A}$ means a set of all actions; $\boldsymbol{P}$ refers to the state transition model, which is used to simulate environmental information. When an action is selected in a state, the information of the next state can be obtained through the model; $\boldsymbol{R}$ is a set of all rewards. γ represents the discount factor for cumulative reward G_t.

State. The state representation contains 6 state features that are designed according to the dynamic and multi-objective characteristics of the problem, which reflect feature information of each sub-action pair in the action space. In the action space, each sub-action pair consists of a candidate operation and a machine, which is equivalent to a candidate action. The set of candidate operations in state s_t is denoted $\boldsymbol{O}_C(s_t)$. It is worth noting that each state feature needs to be converted into a $n \times m$ matrix form.

(1) $p_{ijk}(s_t)$: The processing time of each candidate operation $O_{ij} \in \boldsymbol{O}_C(s_t)$ on all machines in state s_t. If a candidate action consists of a candidate operation and an illegal machine, its processing time is set to a negative value (e.g. −1).

(2) $C_{LB}(O_{ij}, s_t)$: The lower bound of estimated completion time of $O_{ij} \in \boldsymbol{O}_C(s_t)$ in state s_t, as defined in Eq. (8). It is represented as a column vector.

$$C_{LB}(O_{ij}, s_t) = S^E(O_{ij}) + \bar{p}_{ij} \tag{8}$$

where, $S^E(O_{ij})$ is the earliest start time of O_{ij}, as defined in Eq. (9).

$$S^E(O_{ij}) = \max\left\{C^E\left[PJ(O_{ij})\right], C^E\left[PM(O_{ij})\right]\right\} \tag{9}$$

where, there are two items in the maximum operation. The former represents the earliest completion time of $PJ(O_{ij})$, where $PJ(O_{ij})$ is the predecessor operation of O_{ij} on the same job; the latter means the earliest completion time of $PM(O_{ij})$, where $PM(O_{ij})$ is the predecessor operation of O_{ij} on the same machine. If $PJ(O_{ij})$ or $PM(O_{ij})$ does not exist, it is represented by 0.

(3) $R_{LB}(O_{ij}, s_t)$: Assuming that the candidate operation $O_{ij} \in \boldsymbol{O}_C(s_t)$ has been scheduled in state s_t, it means the lower bound of estimated remaining processing time of J_i, which is represented as a column vector. Its calculation formula is as follows:

$$R_{LB}(O_{ij}, s_t) = C_{LB}\left(O_{in_i}, s_t\right) - C_{LB}(O_{ij}, s_t) \tag{10}$$

where, $C_{LB}\left(O_{in_i}, s_t\right)$ stands for the estimated completion time of J_i in state s_t and O_{in_i} represents the last operation of J_i. For the unscheduled operation O_{ij} of J_i, $C_{LB}(O_{ij}, s_t) = C_{LB}(O_{i(j-1)}, s_t) + \bar{p}_{ij}$ is calculated recursively by considering only the priority constraint from its predecessor, i.e. $O_{i(j-1)} \longrightarrow O_{ij}$, and $C_{LB}\left(O_{in_i}, s_t\right) = C_{LB}\left(O_{i(n_i-1)}, s_t\right) + \bar{p}_{in_i}$ if O_{ij} is the last operation O_{in_i} of J_i.

(4) $R_k(s_t)$: The release time of each machine in state s_t, and it is presented as a row vector.

(5) $C_i(s_t)$: The completion time of each job in state s_t is depicted as a column vector. If J_i has not yet started scheduling, then this value represents the random arrival time r_i of the job, reflecting the dynamic characteristic of the problem.

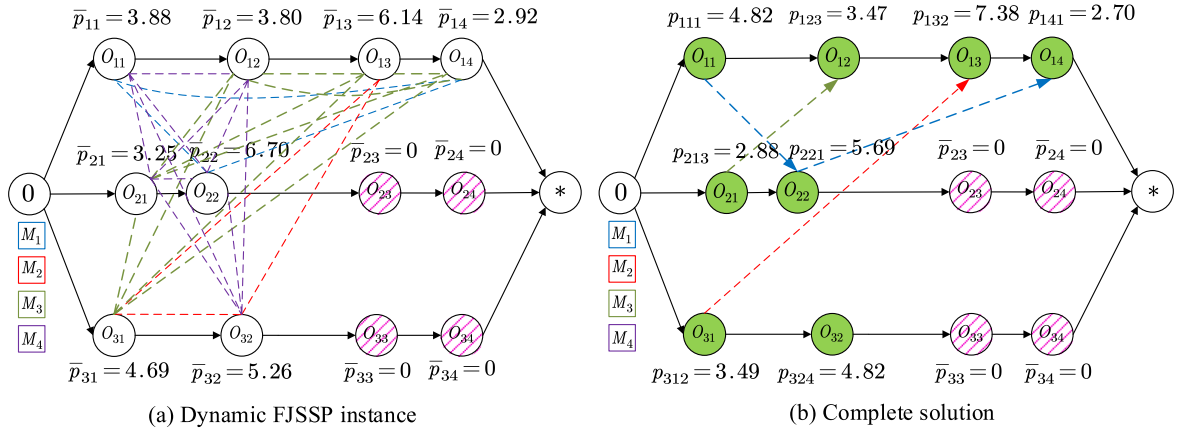


Fig. 1. Disjunction graph representation of the preprocessed DMOFJSSP instance.

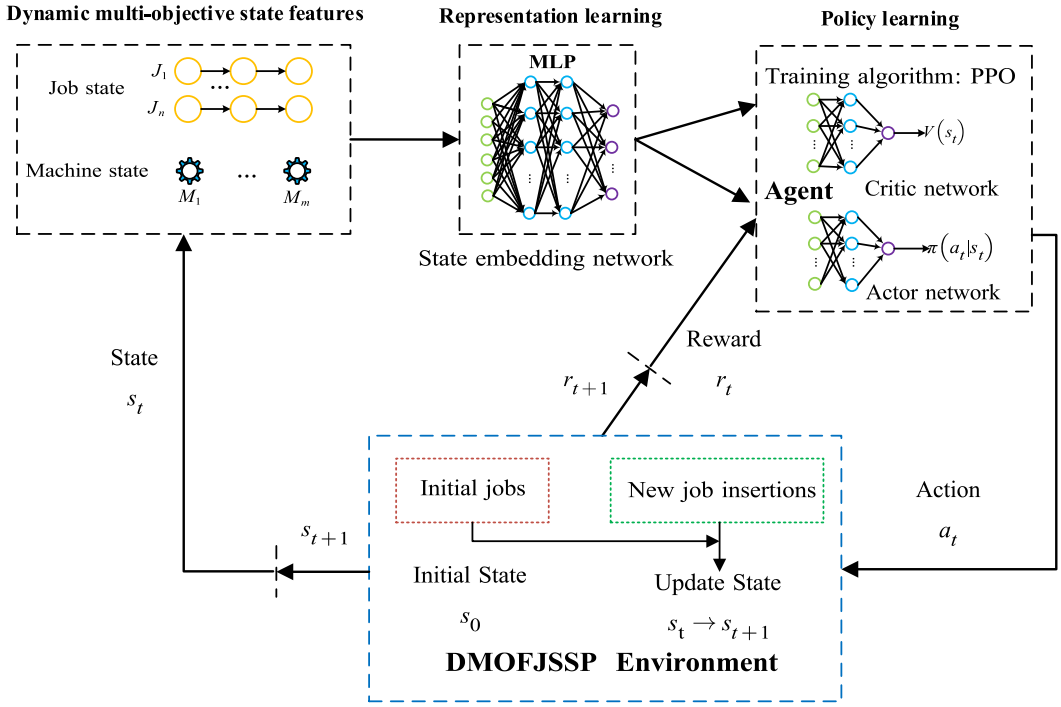


Fig. 2. The DRL framework with reinforcement learning working mechanism.

(6) $D_i(s_t)$: The distance between the $C_i(s_t)$ and the due date $d_i(s_t)$ in state s_t, which is represented as a column vector. Its calculation formula is as follows:

$$D_i(s_t) = |d_i(s_t) - C_i(s_t)| \tag{11}$$

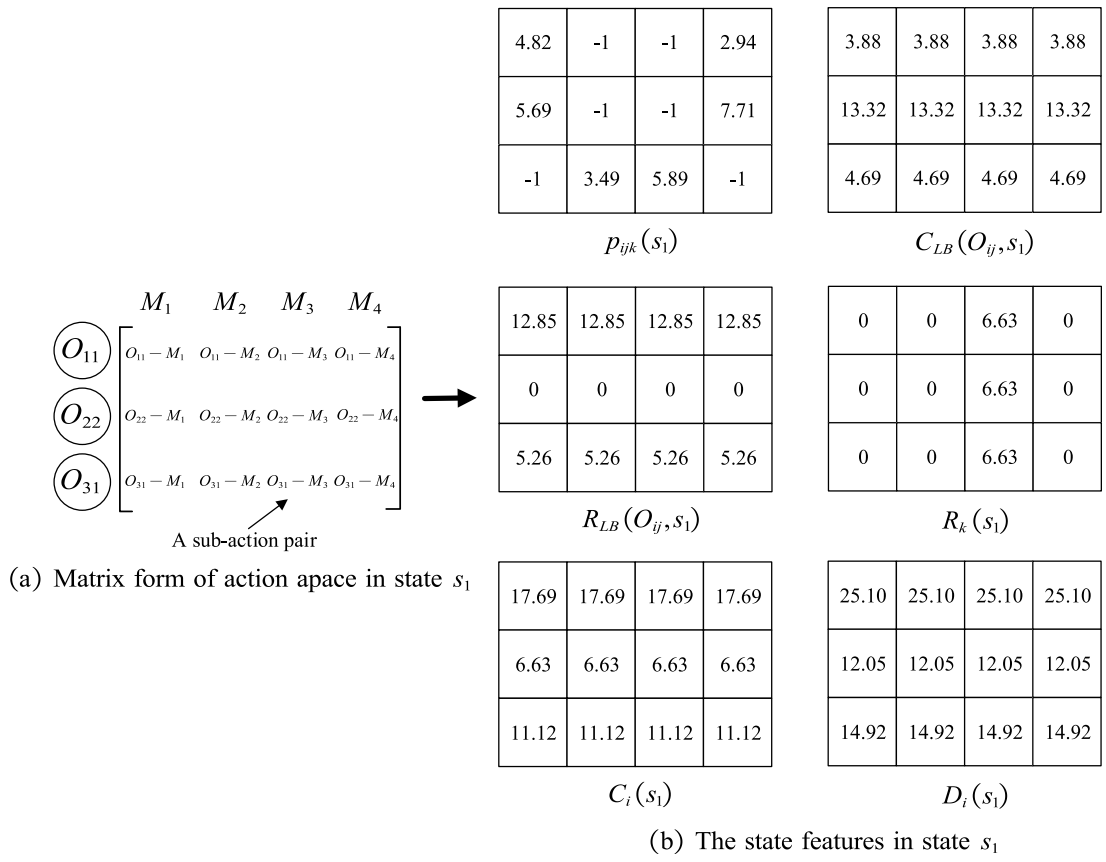
In order to express the state features in matrix form, when the above state features are column vectors, each needs to be expanded into a $n \times m$ matrix by row according to the number of machines m; when the above state features are row vectors, each needs to be expanded into a $n \times m$ matrix by column according to the number of jobs n. Taking the state features of state s_1 as an example, the candidate operation set is $\{O_{11}, O_{22}, O_{31}\}$. Fig. 3 shows an example of state representation in state s_1.

Action. The action space $\boldsymbol{A_t}$ contains some candidate actions, which are represented as sub-action pairs composed of candidate operations and machines. These sub-action pairs form a $n \times m$ matrix. Each candidate action $a_t \in \boldsymbol{A_t}$ represents a sub-action pair, as shown in Fig. 3(a). In the $n \times m$ matrix, the matrix's row coordinates indicate operation information, while its column coordinates represent machine information. During solving, invalid sub-action pairs need to be masked [48]. *Invalid action masking*: The situations where sub-action pairs need to be masked include sub-action pairs composed of candidate operations and illegal machines, as well as the scheduled sub-action pairs. Invalid action masking technology is also widely used in other RL tasks [49–51].

Reward. In order to optimize multiple objectives, the proposed method designs two new reward functions to guide the agent to identify key actions. The design inspiration of two new reward functions comes from the paper [52] and the paper [28], respectively.

The first reward function is designed based on three minimization optimization objectives (makespan, mean lateness and mean flow time), as defined in Eq. (12):

$$r_t = f_{1,t} + f_{2,t} + f_{3,t} \tag{12}$$



(a) Matrix form of action apace in state s_1

(b) The state features in state s_1

Fig. 3. An example of state representation in state s_1.

where, $f_{1,t}$, $f_{2,t}$ and $f_{3,t}$ represent the differences between adjacent decision steps for each optimization objective, respectively. Their calculation formulae are as follows:

$$\begin{cases} f_{1,t} = -\left(F_1\left(s_{t+1}\right) - F_1\left(s_t\right)\right) \\ f_{2,t} = -\left(F_2\left(s_{t+1}\right) - F_2\left(s_t\right)\right) \\ f_{3,t} = -\left(F_3\left(s_{t+1}\right) - F_3\left(s_t\right)\right) \end{cases} \tag{13}$$

We set $\gamma = 1$ and the cumulative reward $G_t = \sum_{t=0}^{T} \gamma^t r_t = F_1(s_0) + F_2(s_0) + F_3(s_0) - (F_1(s_T) + F_2(s_T) + F_3(s_T))$. Since $F_1(s_0)$, $F_2(s_0)$ and $F_3(s_0)$ are all constants, maximizing the cumulative reward G_t and minimizing the three objectives are equivalent.

Algorithm 1 Pseudocode for the second reward function

```
if F2(s_{t+1}) < F2(s_t) or U_ave(s_{t+1}) > 1.005 * U_ave(s_t) then
    r_t=1
else if F2(s_{t+1}) > F2(s_t) or U_ave(s_{t+1}) < U_ave(s_t) then
    r_t=-1
else
    r_t=0
end if
```

The second reward function is designed based on minimizing the mean lateness F_2 and maximizing the average machine utilization U_{ave}. In fact, maximizing U_{ave} is equivalent to minimizing makespan. The design rules of the second reward function are as follows: a positive reward is given when F_2 or U_{ave} becomes better, and a negative reward is given when F_2 or U_{ave} becomes worse. In other cases, the reward is 0. Algorithm 1 is the pseudo code of the second reward function.

4.2. Model training

In the DRL framework, model training includes representation learning and policy learning. Representation learning adopts a 3-layer MLP (excluding the input layer) as the state embedding network. The dimension of the input layer is 6, representing 6 state features, and each state feature input to the network model is represented by a $n \times m$ vector. The dimension of the hidden layer is 128 and the dimension of the output layer is 64. Policy learning uses the network model of the Actor–Critic framework, which includes an actor network and a critic network. The actor network can be used to perform action selection, while the critic network is only used to assist in the training of the actor network. The actor network consists of a 2-layer MLP (excluding the input layer). The input layer has a dimension of 128, the hidden layer has a dimension of 64, and the output layer has a dimension of 1. The critic network is also composed of a 2-layer MLP (excluding the input layer) with an input layer dimension of 64, a hidden layer dimension of 64, and an output layer dimension of 1. The activation function between the network layers is "Tanh".

Fig. 4 shows the working flow chart of the three network models. In Fig. 4, these network models involve two working stages: experience collection stage and update model parameters stage. The first stage is the process of collecting experience from an episode using the old policy, where an episode has T decision steps. The second stage is the process of using the collected experience to update the model parameters by loss function, and this process is used to generate a new policy. It is worth noting that the average pooling in Fig. 4 is to obtain a global representation of all state features, while the extended average pooling operation is to keep the data dimension unchanged so that all state features and their global representation can be concatenated,

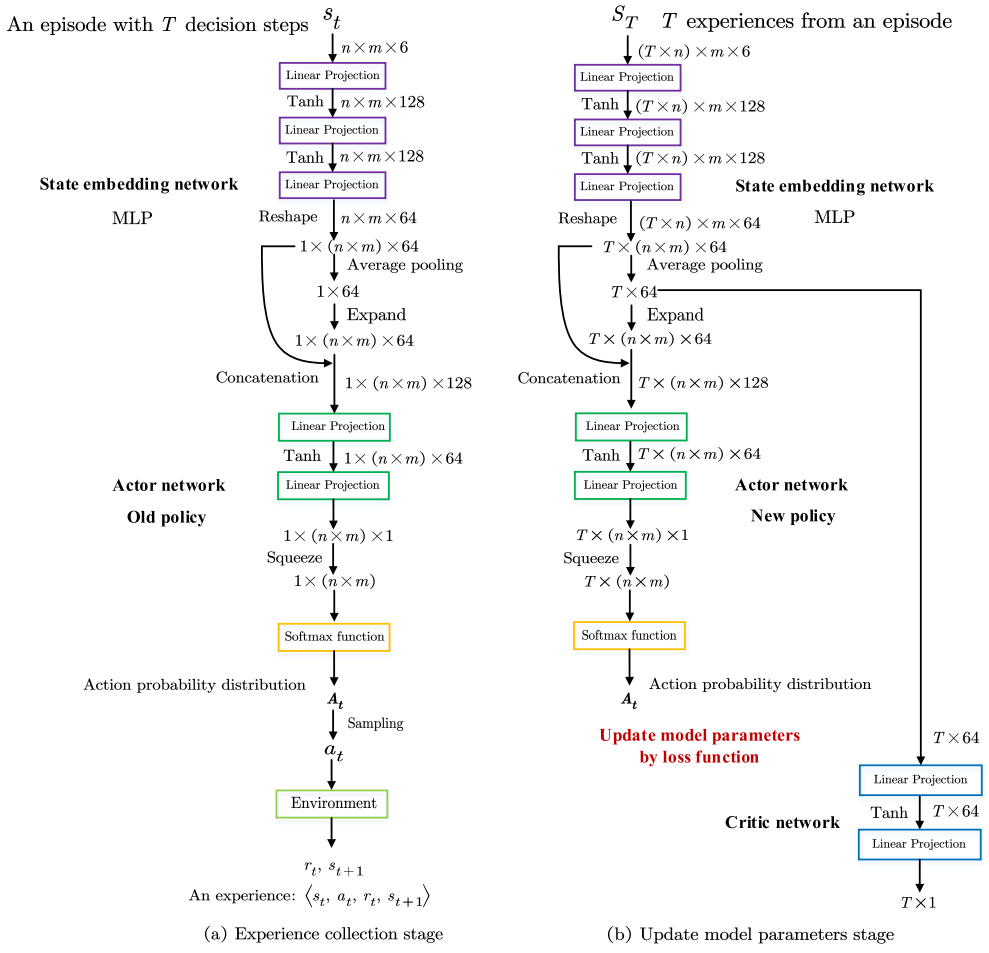


Fig. 4. The working flow chart of the three network models.

Algorithm 2 Pseudocode for training the policy model.

Input: Actor network π_θ with parameter β, behavior actor network $\pi_{\theta_{old}}$ with $\theta_{old}=\theta$, and critic network v_ϕ with ϕ. N, K, V.
Output: θ,ϕ

```
for n = 1, 2, ..., N do
    for t = 1, 2, ..., T do
        while s_t is not terminal do
            Collect the experience ⟨s_t, a_t, r_t, s_{t+1}⟩.
            if s_t is terminal then
                break
            end if
        end while
    end for
    for k = 1, 2, ..., K do
        Calculate L_t(θ, φ) and update θ and φ.
    end for
    if n%V = 0 then
        Performance verification of the policy model.
    end if
end for
```

thus enabling the state embedding network MLP to extract effective state information. In the experience collection stage, after the state information is input into the actor network, an action probability distribution with a $n\times m$ vector is output through the softmax function.

The proximal policy optimization (PPO) algorithm [53] is a kind of policy gradient algorithm, its predecessor is TRPO algorithm [54], and the PPO algorithm has the advantage of easy implementation compared with TRPO algorithm. It shows superior performance in many RL tasks. The PPO algorithm updates the policy model parameters by gradient ascent, and its "surrogate" objective is expressed as follows:

$$L_t^{CLIP}(\theta)=\hat{E}_t\left[\min\left(R_t(\theta)\hat{A}_t,\mathrm{clip}\left(R_t(\theta),1-\epsilon,1+\epsilon\right)\hat{A}_t\right)\right] \tag{14}$$

where,

$$R_t(\theta)=\frac{\pi_\theta\left(a_t\mid s_t\right)}{\pi_{\theta_{old}}\left(a_t\mid s_t\right)} \tag{15}$$

is the ratio of new policy and old policy; $\hat{A}_t$ represents the average advantage brought by action a_t and is denoted as the advantage function; ϵ is used to limit the clipping range of new policy and old policy and is denoted as the clipping coefficient. The total loss function [53] often adds an entropy term and a mean square error term. The entropy term is conducive to improving the exploration ability of the algorithm, and the mean square error term represents the loss function of the critic

Table 2
The hyperparameter settings for PPO algorithm.

Hyperparameters	Values
Number of total episodes N	120, 500
Performance verification every V episodes	2, 20
Number of policy updates K	1
Learning rate lr	1×10^{-3}
Coefficient c_p for policy term	2
Coefficient c_e fo entropy term	0.01
Coefficient c_v for critic term	1
Discount factor γ	1
Clipping coefficient ϵ	0.2

network. Its calculation formula is as follows:

$$\mathcal{L}_t(\theta, \phi) = c_p L_t^{CLIP}(\theta) + c_e {L_t}^E(\theta) - c_v L_t^V(\phi) \tag{16}$$

where, ${L_t}^E(\theta)$ and $L_t^V(\phi)$ represent the entropy term and the mean square error term, respectively. c_p, c_e and c_v represent their corresponding coefficients, respectively. The pseudocode for training the policy model is shown in Algorithm 2. N represents the number of episodes for training; T means the number of decision steps in each episode; Perform K policy updates per collected experience and one-time performance verification on the validation dataset every V episodes.

5. Numerical experiments

5.1. Experimental settings

5.1.1. Configuration

The computing platform: Intel(R) Core(TM) i7-9750H CPU@2.60 GHz 2.59 GHz, and NVIDIA GeForce GTX 1660 Ti with a memory size of 6144 MB for GPU. The software platform: Python version is 3.6.7. PyTorch is available in version 1.7.1, and the principles that drove the implementation of PyTorch and how the key components of the PyTorch runtime work together follow [55]. The hyperparameter settings of the PPO algorithm are shown in Table 2.

5.1.2. Datasets

The datasets contain training dataset, validation dataset and test datasets, where test datasets have two types from different literatures: test dataset 1 [56] and test dataset 2 [32].

Table 3 shows the parameter settings for test dataset 1. In Table 3, the instance scales are as follows: 10×5, 20×5, 50×5, 20×10, 50×10, 100×10, 50×15, 100×15, 200×15. n_i and p_{ijk} are obtained from the corresponding uniform distribution. We set the flexibility rates f to be 1, 0.8, 0.6, and 0.4, and the due date tightness factors c to be 2, 1.5, and 1.2, respectively. The arrival time r_i of each job is obtained according to two different uniform distributions. The estimate due date d_i of each job is calculated by TWK-method [57]. The size of test dataset 1 is 30 for each scale with seed 30.

Table 4 shows the parameter settings for test dataset 2. In Table 4, the instance scales of the new jobs inserted are as follows: 20×8, 20×12, 20×16, 30×8, 30×12, 30×16, 40×8, 40×12, 40×16. The number of initial jobs is 5 for each instance, whose arrival time $r_i = 0$. n_i and p_{ijk} are also obtained from the corresponding uniform distribution. It is assumed that the arrival of the new jobs inserted follows the Poisson distribution, that is, the interval time between the arrival of two adjacent new jobs follows the exponential distribution $exp(1/\lambda)$. For ease of calculation, we set the urgency level (UL for short) of each job to 3 and its corresponding due date tightness (DDT) to 1.5 in an instance. The size of test dataset 2 is 1 for each scale with seed 30.

The training dataset and validation dataset are artificially synthesized according to different distributions. For the training dataset with the uniform distribution, whose parameter settings are from Table 3, where the scale is set to 10×5, the flexibility rate f is set to 1 and the due date tightness factor c is set to 1.5. The size of training dataset with the uniform distribution is 120 with seed 200. For the training dataset with the Poisson distribution, whose parameter settings are from Table 4, where the number of new jobs $n_{New} = 5$, the number of machine $m = 5$, the exponential distribution parameter $\lambda = 20$, and the due date tightness $DDT = 1.2$. The size of training dataset with the Poisson distribution is 500 with seed 200. For the validation dataset, the same validation dataset with the uniform distribution is used for both distributions. The size of validation dataset is 100 with seed 100.

Table 3
The parameter settings for test dataset 1.

Parameters	Values
n	$\{10, 20, 50, 100, 200\}$
m	$\{5, 10, 15\}$
n_i	$U[1, m]$
p_{ijk}	$U[m/2, m \times 2]$
f	1, 0.8, 0.6 and 0.4
c	2, 1.5 and 1.2
r_i	if $n \geq 50$: $U[0, 40]$; otherwise, $U[0, 20]$
d_i	$r_i + c \times \sum_{j=1}^{n_i} \bar{p}_{ij}$

Table 4
The parameter settings for test dataset 2.

Parameters	Values
n_{New}	$\{20, 30, 40\}$
m	$\{8, 12, 16\}$
n_i	$U[1, 20]$
p_{ijk}	$U[1, 50]$
λ	$\{50, 100, 200\}$
DDT	1.5
UL	3

5.1.3. Baselines

The proposed method mainly performs performance comparison with two composite PDRs, three state-of-the-art models and one meta-heuristic method. Among them, the composite PDRs and the state-of-the-art models are all completely reactive scheduling, while the meta-heuristic method belong to predictive reactive scheduling. Doh et al. [10] solved two sub-problems of FJSSP with 36 combination rules, including 12 operation sequencing rules and 3 machine allocation rules. These combination rules contain multiple optimization objectives, such as makespan, total flow time and mean lateness. We choose two combination rules with superior performance to solve DMOFJSSP from the Ref. [10]. These composite PDRs are represented as FIFO+EET and MWKR+EET, which contain two operation sequencing rules (FIFO, MWKR) and one machine allocation rule (EET). Current state-of-the-art models include THDQN [28], DMDDQN [32] and DDQN [33]. The meta-heuristic method uses the Non-Dominated Sorting Genetic Algorithm 3 (NSGA3) [19] that can achieve multi-objective optimization.

- Z_{ij}: priority index of O_{ij}.
- FIFO: index of O_{ij} with the earliest ready time.
- MWKR: max $Z_{ij} = \sum_j^{n_i} \bar{p}_{ij}$.
- EET: index of machine with earliest end time.

5.1.4. Evaluation metrics

Evaluation metrics include optimization capability and real-time. For multi-objective optimization problems, the optimization capability of different methods is measured by some comprehensive evaluation metrics. The comprehensive evaluation metrics used in this paper include Generational Distance (GD) value, Inverse Generational Distance (IGD) value and $\triangle$ value.

(1) Convergence metric: Generational Distance (GD), this metric is used to measure the distance between the non-dominated solution

set $\boldsymbol{A}$ obtained by the algorithm and Pareto optimal solution set $\boldsymbol{P}^*$. Its calculation formula is as follows:

$$GD(\boldsymbol{A}, \boldsymbol{P}^*) = \frac{\sqrt{\sum_{i=1}^{|\boldsymbol{A}|} d_{i,a,p}^2}}{|\boldsymbol{A}|} \tag{17}$$

where, $d_{i,a,p}$ represents the Euclidean distance between the nearest point between each point $a \in \boldsymbol{A}$ and point $p \in \boldsymbol{P}^*$. Considering that the true $\boldsymbol{P}^*$ for the test instances cannot be known in advance, we follow the approach of [28], merge the solutions obtained by the comparison algorithms and take the non-dominated solution set as $\boldsymbol{P}^*$. The smaller the value of GD, the better the convergence of the solution set $\boldsymbol{A}$.

(2) Comprehensive metric: Inverse Generational Distance (IGD), this metric is used to measure the average distance between $\boldsymbol{P}^*$ and $\boldsymbol{A}$. Its calculation formula is as follows:

$$IGD(\boldsymbol{P}^*, \boldsymbol{A}) = \frac{\sum_{i=1}^{|\boldsymbol{P}^*|} d_{i,p,a}}{|\boldsymbol{P}^*|} \tag{18}$$

where, $d_{i,p,a}$ represents the minimum Euclidean distance between point $p \in \boldsymbol{P}^*$ and point $a \in \boldsymbol{A}$. The smaller the value of IGD, the better the comprehensive performance of the solution set $\boldsymbol{A}$ in terms of convergence, distribution uniformity and extensiveness.

(3) Diversity metric: Spread ($\triangle$), this metric is used to measure the unevenness of solution distribution in $\boldsymbol{A}$. Its calculation formula is as follows:

$$\triangle = \frac{\sum_{j=1}^{n_o} d_{j,a,p}^e + \sum_{i=1}^{|\boldsymbol{A}|} |d_{i,a,a} - \bar{d}_{a,a}|}{\sum_{j=1}^{n_o} d_{j,a,p}^e + |\boldsymbol{A}| \bar{d}_{a,a}} \tag{19}$$

where, n_o is the number of optimization objectives, $d_{j,a,p}^e$ represents the Euclidean distance between the boundary solution a in $\boldsymbol{A}$ and the extreme solution of the j objective in $\boldsymbol{P}^*$, $d_{i,a,a}$ represents the Euclidean distance between the i solution in $\boldsymbol{A}$ and the nearest solutions, $\bar{d}_{a,a}$ represents the average value of all $d_{i,a,a}$. The smaller the value of $\triangle$, the better the distribution and diversity of the solution set $\boldsymbol{A}$.

5.2. Experimental results

5.2.1. Training curve of the policy model

Fig. 5 shows training curve of the policy model trained under the uniform distribution with respect to the three optimization objectives. The instance scale of the training dataset is 10×5, and the processing time of each operation ranges from $2.5 \leq p_{ij} \leq 10$ in each instance. In Fig. 5, the two new reward functions designed by the proposed method enable the policy model to show relatively better convergence effects on the three optimization objectives, and the convergence effects of the training curves are basically the same. However, the policy model trained according to the reward of THDQN [28] has relatively poor convergence effect on the latter two optimization objectives (mean lateness and mean flow time), which reflects the effectiveness of the two new reward functions.

Fig. 6 shows training curve of the policy model trained under the Poisson distribution with respect to the three optimization objectives. In Fig. 6, the proposed method uses the second reward to train the policy model under the Poisson distribution, and the training curve of the model shows good convergence.

5.2.2. Comparison to the composite PDRs

To facilitate the performance comparison between different methods, the policy models are trained under different distributions using the second reward. The policy model trained under the uniform distribution is denoted as Ours_U, while the policy model trained under the Poisson distribution is denoted as Ours_P.

For test dataset 1, the proposed method generates a solution through the greedy strategy in the test phase. Table 5 shows the comparison results with composite PDRs on test dataset 1. In Table 5, the solutions obtained by two policy models are better than composite PDRs for three optimization objectives, reflecting the better optimization capability of the policy models. Composite PDRs are easy to implement and have the best real-time performance. However, these rules are often short-sighted, resulting in relatively poor optimization capability.

The f and c used by the proposed method to generate the training dataset are 1 and 1.5, respectively. In order to verify the impact of different combinations of f and c on the generalization of the policy model, four different combinations of test datasets are generated on the maximum instance scale 200×15. These four types of combinations are $f = 1$, $c = 2$; $f = 0.8$, $c = 1.5$; $f = 0.6$, $c = 1.5$; $f = 0.4$, $c = 1.2$. The test dataset for each combination contains 30 instances. Fig. 7 shows the optimization performance of various algorithms on three objectives under different combinations of f and c. It can be seen from Fig. 7 that compared with composite PDRs, the solutions generated by two policy models through generalization show better optimization performance and can effectively dominate the solutions obtained by composite PDRs.

5.2.3. Comparison to the learning-based method and the meta-heuristic method

For test dataset 2, the two policy models trained by the proposed method generate solutions through the sampling strategy instead of the greedy strategy. The current state-of-the-art learning-based methods for solving DMOFJSSP mainly include THDQN [28], DMDDQN [32] and DDQN [33], which mainly involve three optimization objectives: minimizing makespan, maximizing the average machine utilization U_{ave} and minimizing the mean lateness F_2. Among these three optimization objectives, minimizing makespan and maximizing U_{ave} are equivalent, and maximizing U_{ave} can be processed by minimizing $1/U_{ave}$, so it is equivalent to minimizing three optimization objectives. Same as THDQN [28] and DMDDQN [32], each method is repeated independently for 20 times on each instance.

Fig. 8 shows the solution sets obtained by different methods on some representative instances. As can be seen from Fig. 8, compared with other learning-based methods, especially Ours_U, the solution sets generated by the two policy models on instances of different scales have relatively better optimization results, which reflect the superiority of the proposed method.

Table 6 shows the comparison results with learning-based methods and metaheuristic method in terms of GD values. Table 7 shows the comparison results with learning-based methods and meta-heuristic method in terms of IGD values. Table 8 shows the comparison results with learning-based methods and meta-heuristic method in terms of $\triangle$ values. It can be seen from the compared GD, IGD and $\triangle$ values that the solution sets obtained by the proposed method have better convergence, comprehensiveness and diversity on most test instances, especially Ours_U. In addition, NSGA3 also has relatively good convergence and comprehensiveness; however, the diversity of solutions is relatively poor.

In terms of optimization capability, the two policy models trained by the proposed method have better optimization capability than other learning-based methods. This is because the proposed method combines the dynamic and multi-objective characteristics of the problem to design a new state representation, which is beneficial to the agent capture more comprehensive state information to improve its decision-making ability. Different from the previous definition of the action space using multiple PDRs, the new action space definition consists of operation-machine pairs and is combined with invalid action masking technology, which helps the agent to effectively explore the solution space. In addition, the proposed method designs two new reward functions, whose design is closely related to multi-objective optimization and can effectively guide the agent to identify key actions.

Table 9 shows the comparison results with other learning-based methods and the meta-heuristic method in terms of running time. In Table 9, for the learning-based methods, each independent run can generate a complete solution. This time represents the average time of 20 independent runs, while for the meta-heuristic method, it refers to the time to generate a set of solutions, because the meta-heuristic

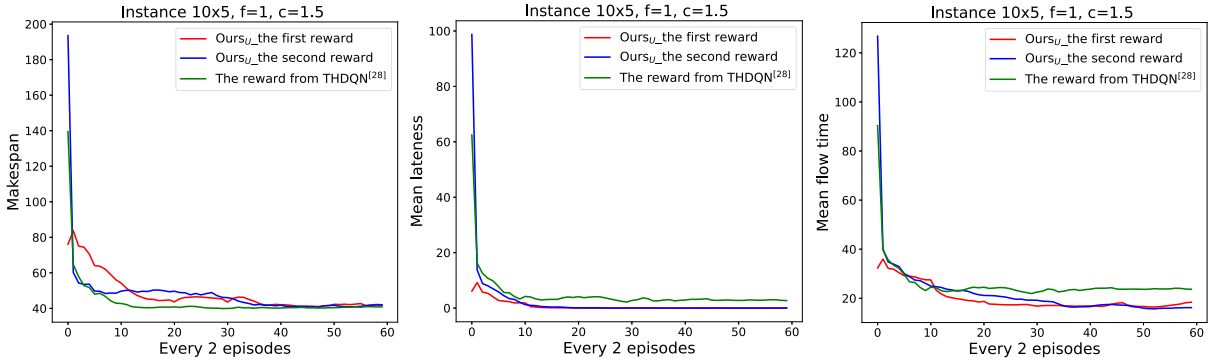


Fig. 5. Training curve of the policy model trained under the uniform distribution.

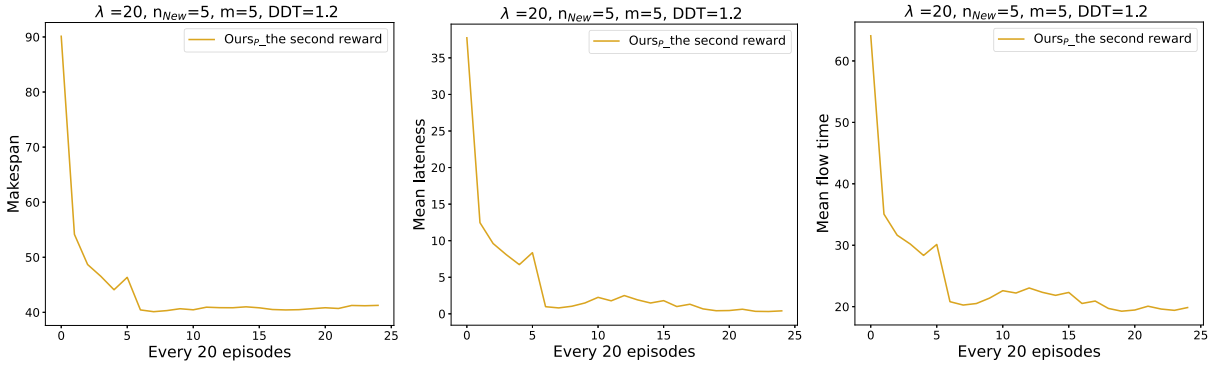


Fig. 6. Training curve of the policy model trained under the Poisson distribution.

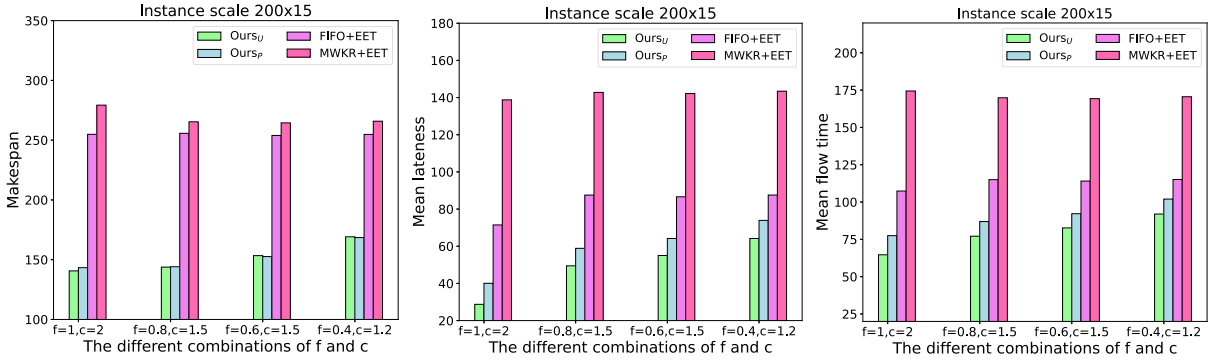
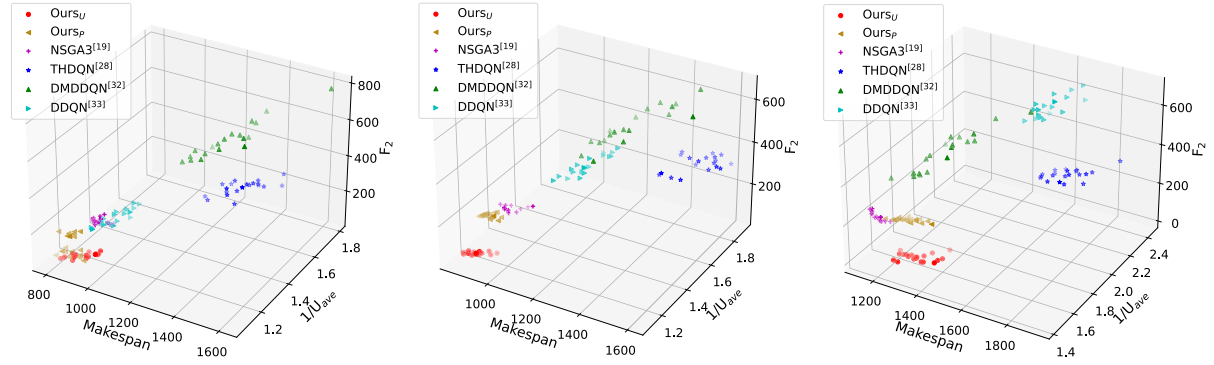


Fig. 7. Performance comparison of various algorithms under different combinations of f and c.



(a) $\lambda = 50, n_{New} = 20, m = 8$ (b) $\lambda = 100, n_{New} = 30, m = 12$ (c) $\lambda = 200, n_{New} = 40, m = 16$

Fig. 8. The solution sets obtained by different methods on some representative instances.

Table 5
Comparison results with composite PDRs.

Instances	$Ours_U$				$Ours_P$				FIFO+EET				MWKR+EET			
$f = 1, c = 1.5$	F_1: Makespan ↓				F_2: Mean lateness↓				F_3: Mean flow time ↓				Time (s): Average time ↓			
$n \times m$	F_1	F_2	F_3	Time (s)	F_1	F_2	F_3	Time (s)	F_1	F_2	F_3	Time (s)	F_1	F_2	F_3	Time (s)
10 × 5	**41.09**	**0.00**	**16.18**	0.09	41.49	0.44	20.57	0.09	52.07	1.11	20.61	0.01	55.42	14.02	35.78	0.01
20 × 5	**59.39**	**1.48**	**25.90**	0.15	62.13	7.05	34.29	0.15	82.55	11.44	35.83	0.02	90.20	34.12	58.12	0.02
50 × 5	**136.00**	**35.24**	**62.69**	0.36	138.08	44.91	72.96	0.37	193.20	52.01	78.57	0.06	216.59	109.70	136.50	0.06
20 × 10	**39.98**	**0.00**	**13.06**	0.16	38.72	0.12	17.53	0.16	55.50	2.94	24.50	0.02	58.91	17.85	41.68	0.02
50 × 10	**68.87**	**0.08**	**20.51**	0.40	75.64	6.15	33.24	0.39	106.25	14.18	39.10	0.07	125.60	52.27	77.92	0.07
100 × 10	**114.37**	**21.65**	**48.25**	0.98	116.72	31.42	59.09	0.99	192.05	51.46	77.97	0.18	212.93	105.88	132.29	0.18
50 × 15	**57.13**	**0.00**	**12.91**	0.46	59.3	0.77	22.89	0.47	81.75	6.35	30.70	0.08	95.32	34.71	59.83	0.08
100 × 15	**77.90**	**1.44**	**25.46**	1.21	84.12	10.72	38.59	1.15	135.35	27.63	54.03	0.20	155.21	70.45	96.70	0.20
200 × 15	**140.45**	**37.16**	**64.53**	3.62	142.21	46.75	74.82	3.41	254.90	79.95	107.38	0.58	279.30	147.19	174.42	0.57

Table 6
Comparison results with learning-based methods and meta-heuristic method in terms of GD values.

λ	n_{New}	m	$Ours_U$	$Ours_P$	NSGA3 [19]	THDQN [28]	DMDDQN [32]	DDQN [33]
50	20	8	1.71e−02	**1.01e−02**	5.45e−02	2.06e−01	2.23e−01	6.82e−02
		12	**2.29e−02**	3.89e−02	5.50e−02	2.58e−01	2.29e−01	8.47e−02
		16	**1.11e−02**	4.98e−02	5.17e−02	2.79e−01	2.24e−01	8.85e−02
100	30	8	**2.18e−02**	3.45e−02	2.80e−02	2.03e−01	2.34e−01	1.26e−01
		12	**6.20e−03**	5.70e−02	6.68e−02	2.63e−01	2.45e−01	1.62e−01
		16	**6.20e−03**	5.18e−02	6.09e−02	2.96e−01	2.22e−01	1.58e−01
200	40	8	**2.49e−02**	7.18e−02	3.94e−02	1.37e−01	2.08e−01	2.60e−01
		12	**1.05e−02**	4.86e−02	4.18e−02	1.82e−01	1.52e−01	2.64e−01
		16	5.80e−03	3.16e−02	**0.00e+00**	1.80e−01	1.22e−01	2.29e−01

Table 7
Comparison results with learning-based methods and meta-heuristic method in terms of IGD values.

λ	n_{New}	m	$Ours_U$	$Ours_P$	NSGA3 [19]	THDQN [28]	DMDDQN [32]	DDQN [33]
50	20	8	4.24e−02	**4.19e−02**	9.30e−02	3.14e−01	3.19e−01	8.09e−02
		12	**0.00e+00**	9.38e−02	1.34e−01	5.47e−01	3.97e−01	1.59e−01
		16	**0.00e+00**	8.33e−02	9.86e−02	4.35e−01	2.74e−01	8.87e−02
100	30	8	5.83e−02	**5.37e−02**	6.29e−02	2.19e−01	2.56e−01	1.47e−01
		12	**0.00e+00**	9.04e−02	1.17e−01	3.97e−01	3.16e−01	2.26e−01
		16	**0.00e+00**	8.23e−02	9.64e−02	4.40e−01	2.73e−01	2.15e−01
200	40	8	7.08e−02	8.93e−02	**3.96e−02**	1.57e−01	1.69e−01	3.19e−01
		12	**4.98e−02**	1.20e−01	9.04e−02	2.52e−01	2.23e−01	3.71e−01
		16	3.87e−02	3.07e−02	**2.90e−02**	1.42e−01	7.79e−02	1.64e−01

Table 8
Comparison results with learning-based methods and meta-heuristic method in terms of △ values.

λ	n_{New}	m	$Ours_U$	$Ours_P$	NSGA3 [19]	THDQN [28]	DMDDQN [32]	DDQN [33]
50	20	8	7.00e−01	7.30e−01	9.75e−01	7.63e−01	8.54e−01	**6.63e−01**
		12	**7.15e−01**	8.02e−01	9.18e−01	8.06e−01	8.43e−01	7.28e−01
		16	7.08e−01	7.46e−01	1.04e+00	8.28e−01	8.39e−01	**6.79e−01**
100	30	8	7.76e−01	8.07e−01	1.13e+00	**7.60e−01**	8.46e−01	7.94e−01
		12	**6.43e−01**	7.71e−01	1.00e+00	8.17e−01	7.78e−01	8.16e−01
		16	**7.43e−01**	8.32e−01	1.10e+00	8.40e−01	8.27e−01	7.88e−01
200	40	8	**6.82e−01**	8.09e−01	1.04e+00	7.72e−01	7.89e−01	8.20e−01
		12	**7.27e−01**	8.35e−01	1.04e+00	7.37e−01	8.49e−01	8.39e−01
		16	**7.79e−01**	7.95e−01	9.83e−01	8.39e−01	8.38e−01	8.61e−01

method improves the quality of the solution through continuous iterative search, the time to generate a complete solution is the same as the time to generate a set of solutions. As can be seen from Table 9, the two policy models trained by the proposed method have a faster solution speed than other learning-based methods and the meta-heuristic method.

In terms of real-time performance, compared with other learning-based methods, the proposed method adopts a lightweight network architecture to implement representation learning and policy learning, so it has lower computational complexity. In addition, the new action space definition means that only one policy model needs to be trained to solve the operation sequencing sub-problem and the machine allocation sub-problem at the same time, thus reducing the computational complexity of the algorithm to some extent, reflecting the better applicability of the model. However, current other learning-based methods often use two DQNs or hierarchical DQNs to solve DMOFJSSP, which increases the computational complexity of the network model. Since meta-heuristic method improves the quality of the solution through continuous iterative search, it belongs to predictive-reactive scheduling, and this type of method is not suitable for real-time scheduling.

In short, PDRs are easy to implement, so they have the best real-time performance. However, due to the short-sighted defect of PDRs, their optimization capabilities are relatively poor. Meta-heuristic method also has good optimization capability, but this type of method belongs to predictive reactive scheduling and its real-time performance is poor. In the acceptable time complexity, the two policy models trained the proposed method have better optimization performance than composite

Table 9
Comparison results with learning-based methods and meta-heuristic method in terms of running time.

λ	n_{New}	m	Ours$_U$	Ours$_P$	NSGA3 [19]	THDQN [28]	DMDDQN [32]	DDQN [33]
50	20	8	**0.59**	0.67	98.46	14.71	14.51	5.72
		12	**0.62**	0.64	100.63	14.78	14.67	5.99
		16	**0.65**	0.69	101.95	14.87	14.71	5.76
100	30	8	**0.89**	0.94	158.90	21.34	20.70	8.46
		12	**0.96**	0.98	159.25	31.37	20.99	8.66
		16	**1.01**	1.05	161.05	21.78	21.06	8.60
200	40	8	**1.19**	1.21	213.52	27.78	27.04	10.96
		12	**1.26**	1.30	212.07	27.80	27.38	10.92
		16	**1.36**	1.37	215.06	27.83	27.38	11.35

PDRs. Compared with other learning-based methods, the policy models trained the proposed method not only have better optimization capability, but also have the lowest computational complexity, which reflect better real-time performance.

6. Conclusion

We solve DMOFJSSP via DRL in this paper. The dynamic event of this problem is new job insertions. The proposed method designs a new MDP model, in which the new state representation is designed according to the dynamic and multi-objective characteristics of the problem, which is beneficial for the agent to capture more comprehensive state information to improve its decision-making ability. Different from the previous definition of action space through multiple PDRs, the new action space is composed of sub-action pairs composed of operations and machines, which helps the agent to better explore the solution space; two new reward function designs are closely related to multi-objective optimization and can effectively guide the agent to identify key actions. In addition, the proposed method uses a lightweight network architecture to implement representation learning and policy learning, which reduces the computational complexity of the algorithm to some extent. Finally, we evaluate the generalization on two test datasets from different literatures. Experimental results show that the two policy models trained by the proposed method have better optimization capability than the well-known composite PDRs; comparison with the current state-of-the-art models in terms of GD, IGD and $\triangle$ values, the solution sets obtained by the proposed method have better convergence, comprehensiveness and diversity on most test instances. In addition, compared with the current state-of-the-art models, the two policy models trained by the proposed method have lower computational complexity and reflect better applicability.

CRediT authorship contribution statement

Erdong Yuan: Writing – original draft, Software, Conceptualization. **Liejun Wang:** Writing – review & editing, Supervision, Funding acquisition. **Shiji Song:** Visualization, Resources, Funding acquisition. **Shuli Cheng:** Writing – review & editing, Visualization, Formal analysis. **Wei Fan:** Resources, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was funded in part by the Scientific and technological innovation 2030 major project under Grant 2022ZD0115800, the Xinjiang Uygur Autonomous Region Tianshan Excellence Project, China under grant 2022TSYCLJ0036, the National Science Foundation of China under Grant U1903213 and 72322021, the Key Project of the National Natural Science Foundation of China under Grant 61936009, Natural Science Foundation of Xinjiang Uygur Autonomous Region under Grant 2022D01C82, and Xinjiang Uygur Autonomous Region Postgraduate Research and Innovation Project, China under Grant XJ2022G025.

Data availability

Data will be made available on request.

References

[1] Amir Rajabinasab, Saeed Mansour, Dynamic flexible job shop scheduling with alternative process plans: an agent-based approach, Int. J. Adv. Manuf. Technol. 54 (2011) 1091–1107.
[2] Paolo Brandimarte, Routing and scheduling in a flexible job shop by tabu search, Ann. Oper. Res. 41 (3) (1993) 157–183.
[3] Ferdinando Pezzella, Gianluca Morganti, Giampiero Ciaschetti, A genetic algorithm for the flexible job-shop scheduling problem, Comput. Oper. Res. 35 (10) (2008) 3202–3212.
[4] Imran Ali Chaudhry, Abid Ali Khan, A research survey: review of flexible job shop scheduling techniques, Int. Trans. Oper. Res. 23 (3) (2016) 551–591.
[5] Yuan Yuan, Hua Xu, Multiobjective flexible job shop scheduling using memetic algorithms, IEEE Trans. Autom. Sci. Eng. 12 (1) (2013) 336–353.
[6] Djamila Ouelhadj, Sanja Petrovic, A survey of dynamic scheduling in manufacturing systems, J. Sched. 12 (2009) 417–431.
[7] KM.M.A. Bukkur, M.I. Shukri, Osama Mohammed Elmardi, A review for dynamic scheduling in manufacturing, Glob. J. Res. Eng. 18 (5-J) (2018) 25–37.
[8] Bruno Cunha, Ana M Madureira, Benjamim Fonseca, Duarte Coelho, Deep reinforcement learning as a job shop scheduling solver: A literature review, in: Hybrid Intelligent Systems: 18th International Conference on Hybrid Intelligent Systems (HIS 2018) Held in Porto, Portugal, December 13-15, 2018 18, Springer, 2020, pp. 350–359.
[9] Yazid Mati, Nidhal Rezg, Xiaolan Xie, An integrated greedy heuristic for a flexible job shop scheduling problem, in: 2001 IEEE International Conference on Systems, Man and Cybernetics. E-Systems and E-Man for Cybernetics in Cyberspace (Cat. No. 01CH37236), Vol. 4, IEEE, 2001, pp. 2534–2539.
[10] Hyoung-Ho Doh, Jae-Min Yu, Ji-Su Kim, Dong-Ho Lee, Sung-Ho Nam, A priority scheduling approach for flexible job shops with multiple process plans, Int. J. Prod. Res. 51 (12) (2013) 3748–3764.
[11] Xiao-Ning Shen, Xin Yao, Mathematical modeling and multi-objective evolutionary algorithms applied to dynamic flexible job shop scheduling problems, Inform. Sci. 298 (2015) 198–224.
[12] Kai Zhou Gao, Ponnuthurai Nagaratnam Suganthan, Tay Jin Chua, Chin Soon Chong, Tian Xiang Cai, Qan Ke Pan, A two-stage artificial bee colony algorithm scheduling flexible job-shop scheduling problem with new job insertion, Expert Syst. Appl. 42 (21) (2015) 7652–7663.
[13] Maroua Nouiri, Abdelghani Bekrar, Abderrazak Jemai, Damien Trentesaux, Ahmed Chiheb Ammari, Smail Niar, Two stage particle swarm optimization to solve the flexible job shop predictive scheduling problem considering possible machine breakdowns, Comput. Ind. Eng. 112 (2017) 595–606.
[14] Kexin Li, Qianwang Deng, Like Zhang, Qing Fan, Guiliang Gong, Sun Ding, An effective MCTS-based algorithm for minimizing makespan in dynamic flexible job shop scheduling problem, Comput. Ind. Eng. 155 (2021) 107211.
[15] Cameron B Browne, Edward Powley, Daniel Whitehouse, Simon M Lucas, Peter I Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, Simon Colton, A survey of monte carlo tree search methods, IEEE Trans. Comput. Intell. AI Games 4 (1) (2012) 1–43.
[16] Guilherme E. Vieira, Jeffrey W. Herrmann, Edward Lin, Rescheduling manufacturing systems: a framework of strategies, policies, and methods, J. Sched. 6 (2003) 39–62.

[17] Ripon K. Chakrabortty, Ruhul A. Sarker, Daryl L. Essam, Multi-mode resource constrained project scheduling under resource disruptions, Comput. Chem. Eng. 88 (2016) 13–29.

[18] Youjun An, Xiaohui Chen, Kaizhou Gao, Yinghe Li, Lin Zhang, Multiobjective flexible job-shop rescheduling with new job insertion and machine preventive maintenance, IEEE Trans. Cybern. 53 (5) (2023) 3101–3113.

[19] Himanshu Jain, Kalyanmoy Deb, An evolutionary many-objective optimization algorithm using reference-point based nondominated sorting approach, part II: Handling constraints and extending to an adaptive approach, IEEE Trans. Evol. Comput. 18 (4) (2013) 602–622.

[20] Lixin Wei, Jinxian He, Zeyin Guo, Ziyu Hu, A multi-objective migrating birds optimization algorithm based on game theory for dynamic flexible job shop scheduling problem, Expert Syst. Appl. 227 (2023) 120268.

[21] Jian Xiong, Li-ning Xing, Ying-wu Chen, Robust scheduling for multi-objective flexible job-shop problems with random machine breakdowns, Int. J. Prod. Econ. 141 (1) (2013) 112–126.

[22] Jiae Zhang, Jianjun Yang, Yong Zhou, Robust scheduling for multi-objective flexible job-shop problems with flexible workdays, Eng. Optim. 48 (11) (2016) 1973–1989.

[23] Xiao-Ning Shen, Ying Han, Jing-Zhi Fu, Robustness measures and robust scheduling for multi-objective stochastic flexible job shop scheduling problems, Soft Comput. 21 (2017) 6531–6554.

[24] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, Martin Riedmiller, Playing atari with deep reinforcement learning, 2013, arXiv preprint arXiv:1312.5602.

[25] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al., Mastering chess and shogi by self-play with a general reinforcement learning algorithm, 2017, arXiv preprint arXiv:1712.01815.

[26] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al., Grandmaster level in StarCraft II using multi-agent reinforcement learning, Nature 575 (7782) (2019) 350–354.

[27] Shu Luo, Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning, Appl. Soft Comput. 91 (2020) 106208.

[28] Shu Luo, Linxuan Zhang, Yushun Fan, Dynamic multi-objective scheduling for flexible job shop by deep reinforcement learning, Comput. Ind. Eng. 159 (2021) 107489.

[29] Shu Luo, Linxuan Zhang, Yushun Fan, Real-time scheduling for dynamic partial-no-wait multiobjective flexible job shop by deep reinforcement learning, IEEE Trans. Autom. Sci. Eng. 19 (4) (2021) 3020–3038.

[30] Renke Liu, Rajesh Piplani, Carlos Toro, Deep reinforcement learning for dynamic scheduling of a flexible job shop, Int. J. Prod. Res. 60 (13) (2022) 4049–4069.

[31] Hado Van Hasselt, Arthur Guez, David Silver, Deep reinforcement learning with double q-learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 30, 2016.

[32] Hao Wang, Junfu Cheng, Chang Liu, Yuanyuan Zhang, Shunfang Hu, Liangyin Chen, Multi-objective reinforcement learning framework for dynamic flexible job shop scheduling problem with uncertain events, Appl. Soft Comput. 131 (2022) 109717.

[33] Jingru Chang, Dong Yu, Yi Hu, Wuwei He, Haoyu Yu, Deep reinforcement learning for dynamic flexible job shop scheduling with random job arrival, Processes 10 (4) (2022) 760.

[34] Lu Zhang, Yi Feng, Qinge Xiao, Yunlang Xu, Di Li, Dongsheng Yang, Zhile Yang, Deep reinforcement learning for dynamic flexible job shop scheduling problem considering variable processing times, J. Manuf. Syst. 71 (2023) 257–273.

[35] Yong Gui, Dunbing Tang, Haihua Zhu, Yi Zhang, Zequn Zhang, Dynamic scheduling for flexible job shop using a deep reinforcement learning approach, Comput. Ind. Eng. 180 (2023) 109255.

[36] David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, Martin Riedmiller, Deterministic policy gradient algorithms, in: International Conference on Machine Learning, Pmlr, 2014, pp. 387–395.

[37] Yun Liu, Fangfang Zhang, Yanan Sun, Mengjie Zhang, Evolutionary trainer-based deep Q-network for dynamic flexible job shop scheduling, IEEE Trans. Evol. Comput. (2024) 1.

[38] Hongtao Tang, Yu Xiao, Wei Zhang, Deming Lei, Jing Wang, Tao Xu, A DQL-NSGA-III algorithm for solving the flexible job shop dynamic scheduling problem, Expert Syst. Appl. 237 (2024) 121723.

[39] Yong Zhou, Jian-jun Yang, Zhuang Huang, Automatic design of scheduling policies for dynamic flexible job shop scheduling via surrogate-assisted cooperative co-evolution genetic programming, Int. J. Prod. Res. 58 (9) (2020) 2561–2580.

[40] Fangfang Zhang, Yi Mei, Su Nguyen, Mengjie Zhang, Evolving scheduling heuristics via genetic programming with feature selection in dynamic flexible job-shop scheduling, Ieee Trans. Cybern. 51 (4) (2020) 1797–1811.

[41] Fangfang Zhang, Yi Mei, Su Nguyen, Mengjie Zhang, Correlation coefficient-based recombinative guidance for genetic programming hyperheuristics in dynamic flexible job shop scheduling, IEEE Trans. Evol. Comput. 25 (3) (2021) 552–566.

[42] Fangfang Zhang, Yi Mei, Su Nguyen, Mengjie Zhang, Kay Chen Tan, Surrogate-assisted evolutionary multitask genetic programming for dynamic flexible job shop scheduling, IEEE Trans. Evol. Comput. 25 (4) (2021) 651–665.

[43] Meng Xu, Yi Mei, Fangfang Zhang, Mengjie Zhang, Genetic programming and reinforcement learning on learning heuristics for dynamic scheduling: A preliminary comparison, IEEE Comput. Intell. Mag. 19 (2) (2024) 18–33.

[44] Parviz Fattahi, Mohammad Saidi Mehrabad, Fariborz Jolai, Mathematical modeling and heuristic approaches to flexible job shop scheduling problems, J. Intell. Manuf. 18 (2007) 331–342.

[45] Jacek Błażewicz, Erwin Pesch, Małgorzata Sterna, The disjunctive graph machine representation of the job shop scheduling problem, European J. Oper. Res. 127 (2) (2000) 317–331.

[46] Scott J. Mason, Kasin Oey, Scheduling complex job shops using disjunctive graphs: a cycle elimination procedure, Int. J. Prod. Res. 41 (5) (2003) 981–994.

[47] Ruiqi Chen, Wenxin Li, Hongbing Yang, A deep reinforcement learning framework based on an attention mechanism and disjunctive graph embedding for the job-shop scheduling problem, IEEE Trans. Ind. Inform. 19 (2) (2022) 1322–1331.

[48] Erdong Yuan, Liejun Wang, Shuli Cheng, Shiji Song, Wei Fan, Yongming Li, Solving flexible job shop scheduling problems via deep reinforcement learning, Expert Syst. Appl. 245 (2024) 123019.

[49] Shengyi Huang, Santiago Ontañón, A closer look at invalid action masking in policy gradient algorithms, 2020, arXiv preprint arXiv:2006.14171.

[50] Yueqi Hou, Xiaolong Liang, Jiaqiang Zhang, Qisong Yang, Aiwu Yang, Ning Wang, Exploring the use of invalid action masking in reinforcement learning: A comparative study of on-policy and off-policy algorithms in real-time strategy games, Appl. Sci. 13 (14) (2023) 8283.

[51] Bolei Chen, Ping Zhong, Yongzheng Cui, Siyi Lu, Yixiong Liang, Yu Sheng, EMExplorer: an episodic memory enhanced autonomous exploration strategy with voronoi domain conversion and invalid action masking, Complex Intell. Syst. (2023) 1–15.

[52] Cong Zhang, Wen Song, Zhiguang Cao, Jie Zhang, Puay Siew Tan, Xu Chi, Learning to dispatch for job shop scheduling via deep reinforcement learning, Adv. Neural Inf. Process. Syst. 33 (2020) 1621–1632.

[53] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov, Proximal policy optimization algorithms, 2017, arXiv preprint arXiv:1707.06347.

[54] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, Philipp Moritz, Trust region policy optimization, in: International Conference on Machine Learning, PMLR, 2015, pp. 1889–1897.

[55] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al., Pytorch: An imperative style, high-performance deep learning library, Adv. Neural Inf. Process. Syst. 32 (2019).

[56] Joc Cing Tay, Nhu Binh Ho, Evolving dispatching rules using genetic programming for solving multi-objective flexible job-shop problems, Comput. Ind. Eng. 54 (3) (2008) 453–473.

[57] Kenneth R. Baker, Sequencing rules and due-date assignments in a job shop, Manage. Sci. 30 (9) (1984) 1093–1104.